

计算机视觉第一次作业--全景图拼接

20337025 崔璨明

1. 实验目的

1. 熟悉 Harris 角点检测器的原理和基本使用
2. 熟悉 RANSAC 抽样一致方法的使用场景
3. 熟悉 HOG 描述子的基本原理

2. 实验要求

1. 提交实验报告，要求有适当步骤说明和结果分析，对比
2. 将代码和结果打包提交
3. 实验可以使用现有的特征描述子实现

3. 实验内容

1. 使用 Harris 焦点检测器寻找关键点。
2. 构建描述算子来描述图中的每个关键点，比较两幅图像的两组描述子，并进行匹配。
3. 根据一组匹配关键点，使用 RANSAC 进行仿射变换矩阵的计算。
4. 将第二幅图变换过来并覆盖在第一幅图上，拼接形成一个全景图像。
5. 实现不同的描述子，并得到不同的拼接结果。

4. 实验过程

4-1 Harris 角点算法

4-1-1 实验原理

Harris 角点检测算法的基本思想是通过计算图像灰度值的变化率来检测图像中的角点。该算法的核心是计算图像中每个像素点的 Harris 值，Harris 值越大则该像素点越可能是角点。

具体来说，对于图像中的每个像素点，可以通过以下步骤计算其 Harris 值：

1. 计算该像素点在 x 和 y 方向的梯度分别为 I_x 和 I_y 。
2. 根据 I_x 和 I_y 计算出该像素点的结构张量 M ：

$$M = \begin{bmatrix} \sum_{x,y} w(x,y) I_x^2 & \sum_{x,y} w(x,y) I_x I_y \\ \sum_{x,y} w(x,y) I_x I_y & \sum_{x,y} w(x,y) I_y^2 \end{bmatrix}$$

其中， $w(x,y)$ 是一个窗口函数，用于对图像进行局部加权，一般为高斯函数或矩形函数。

3. 计算 M 的特征值 λ_1 和 λ_2 ，即：

$$\lambda_1, \lambda_2 = \frac{1}{2} \left[\text{tr}(M) \pm \sqrt{\text{tr}(M)^2 - 4 \det(M)} \right]$$

其中， $\text{tr}(M) = \sum_{i=1}^2 M_{ii}$ ， $\det(M) = M_{11}M_{22} - M_{12}^2$ 。

4. 计算 Harris 值 H ： $H = k\lambda_1\lambda_2 - k(\lambda_1 + \lambda_2)^2$ 其中， k 是一个常数，一般取值为 0.04 到 0.06。

根据计算出来的 Harris 值，可以通过阈值筛选的方法得到角点。一般来说，取 Harris 值最大的 n 个像素点作为角点。

4-1-2 具体实现与代码展示

主要运用了 `opencv` 库进行实现，首先我们载入图像，计算图像梯度——利用 Sobel 滤波器对输入图像进行卷积，得到图像在 x 和 y 方向上的梯度值。然后计算结构张量的各个元素，利用图像的梯度计算二阶导数，并组合成一个 2×2 的结构张量矩阵。接着对结构张量的各个元素进行高斯加权平均，主要调用了 `getGaussianKernel` 方法，利用高斯核对结构张量的各个元素进行加权平均，得到加权平均后的结构张量。接着对加权平均后的结构张量计算 Harris 值。

得到 Harris 值后，对于每个像素点，比较其 Harris 值和周围像素点的 Harris 值，如果是最大的，则将其标记为角点（若角点数量超过最大值，则只选取 Harris 值最大的前 `max_corners` 个角点）。

关键代码如下：

```
def detect_corners(img):
    # 计算梯度
    dx = cv2.Sobel(img, cv2.CV_64F, 1, 0, ksize=SIZE)
    dy = cv2.Sobel(img, cv2.CV_64F, 0, 1, ksize=SIZE)
    # 计算结构张量
    dx2 = dx * dx
    dy2 = dy * dy
    dxy = dx * dy
    # 高斯加权平均
    weights = cv2.getGaussianKernel(SIZE, sigma)
    sx2 = cv2.filter2D(dx2, -1, weights)
    sy2 = cv2.filter2D(dy2, -1, weights)
    sxy = cv2.filter2D(dxy, -1, weights)
    # 计算 Harris 值
    detM = sx2 * sy2 - sxy * sxy
    traceM = sx2 + sy2
    Harris = k * detM / (traceM + 1e-12)
    # 选取 Harris 值最大的像素点作为角点
    corner_list = []
    offset = SIZE // 2
    for i in range(offset, img.shape[0] - offset):
        for j in range(offset, img.shape[1] - offset):
            if Harris[i, j] > threshold and Harris[i, j] == np.max(Harris[i-
offset:i+offset+1, j-offset:j+offset+1]):
                corner_list.append((j, i))
    # 如果角点数量超过最大值，则选前 maxnum 个角点
    if len(corner_list) > maxnum:
        corner_list = sorted(corner_list, key=lambda x: Harris[x[1], x[0]],
reverse=True)[:maxnum]
    return corner_list
```

得到角点的列表后，进行绘制和保存：

```
for corner in corner_list:
    cv2.circle(img, corner, 3, (0, 0, 255), 2)
cv2.imwrite('results/1/sudoku_keypoints.png', img)
```

4-2 关键点描述与匹配

4-2-1 实验原理

SIFT (Scale Invariant Feature Transform) 特征描述子是一种图像特征提取和匹配算法，可用于在两个或多个图像之间寻找对应的关键点。SIFT特征点匹配包括两个关键步骤：特征提取和特征匹配。

在特征提取阶段，SIFT算法通过尺度空间中的高斯差分金字塔来检测图像中的极值点，对每个极值点周围的像素进行方向分配和特征描述，形成SIFT特征向量。

在特征匹配阶段，SIFT算法使用基于特征向量之间的欧几里得距离的匹配方法来确定两个图像之间的特征点的对应关系。通常可以使用暴力匹配方法来快速匹配特征点。

HOG (Histogram of Oriented Gradients) 算法的基本思想是将图像划分成小的单元格，计算每个单元格的像素梯度的方向直方图，然后将所有单元格的直方图串联起来，形成一个特征向量，用于表示整个图像。

在使用HOG特征作为描述子进行**关键点匹配**中，我们首先需要使用HOG算法提取两幅图像中的特征点，并计算每个特征点的HOG特征向量。然后，我们使用一种匹配算法（例如SIFT，SURF或ORB）来比较两幅图像中的特征点，并找到最佳匹配。

RANSAC (Random Sample Consensus) 也称随机一致性采样。该算法是用来求解单应性矩阵并找到正确的直线来拟合带有噪声的数据。也就是说，给定一组数据点集中包含正确的点和噪声点，RANSAC能找到一条直线来挑选出正确的点，同时摒弃噪声点。基于RANSAC算法的图像拼接是利用RANSAC算法估计出的图像间的单应性矩阵，再将所有图像扭曲到一个公共的图像平面上，利用RANSAC算法进行图像拼接是为了减少多幅图像之间出现的特征点错配问题。

对于实现**图像拼接**方面，分别使用 SIFT 特征和 HOG 特征作为描述子获得两幅图像的关键点的特征，通过检测两张图片中的特征点并计算特征描述子，然后对特征点进行匹配，并找出匹配的对应点，通过计算这些对应点之间的变换关系 (Homography)，最终将两张图片拼接在一起。

4-2-2 具体实现与代码展示

我编写了一个 `Stitcher` 类来进行所有的工作，包括两种特种子的关键点匹配和图像拼接。

SIFT特征描述子实现关键点匹配可以直接调用 `SIFT_create()` 方法创建SIFT对象，然后通过 `detectAndCompute()` 方法将计算灰度图的关键点和特征。接着创建一个BFMatcher对象，并使用它来匹配两个图像的描述符，根据特征点匹配程度对匹配结果进行排序。最后绘制匹配结果图像，使用 `cv2.drawMatches` 函数将匹配结果可视化。关键代码实现如下：

```
def match(self):
    # 创建SIFT对象
    sift = cv2.SIFT_create()
    gray1 = cv2.cvtColor(self.img2, cv2.COLOR_BGR2GRAY)
    gray2 = cv2.cvtColor(self.img1, cv2.COLOR_BGR2GRAY)
    # 检测关键点和计算描述符
    kp1, des1 = sift.detectAndCompute(gray1, None)
    kp2, des2 = sift.detectAndCompute(gray2, None)
    # 创建BFMatcher对象
    bf = cv2.BFMatcher(cv2.NORM_L2, crossCheck=True)
    # 匹配关键点描述符
    matches = bf.match(des1, des2)
    # print(matches)
    # 根据特征点匹配程度排序
    matches = sorted(matches, key = lambda x:x.distance)
    # 绘制匹配结果
```

```

# print(matches)
result =
cv2.drawMatches(self.img2, kp1, self.img1, kp2, matches[:, None], flags=cv2.DrawMatchesFlags_NOT_DRAW_SINGLE_POINTS)
# 保存匹配结果图像
cv2.imwrite("results/1/uttower_match.png", result)

```

HOG特征描述子关键点匹配较为复杂，需要利用到SIFT的算法匹配的特征点，具体思路为遍历这些特征点，如果当前关键点周围有足够的像素，则从图像中提取一个64x64大小的图像块(patch)，并使用HOGDescriptor对象计算该块的HOG特征描述符，保存起来。对于图像1列表中的每个HOG特征描述符，计算其与图像2列表中所有特征描述符的距离，然后找到与其距离最近的一个描述符，将其添加到matches列表中，直到所有特征点都匹配到列表中，关键代码实现如下：

```

def HOG_kps_match(self):
    # 获取输入的特征点
    kp1=self.kp1
    kp2=self.kp2
    corners1 = []
    corners2 = []
    for kp in kp1:
        x, y = int(kp.pt[0]), int(kp.pt[1])
        corners1.append((x,y))
    for kp in kp2:
        x, y = int(kp.pt[0]), int(kp.pt[1])
        corners2.append((x,y))
    #参数
    winSize = (64, 64)
    blockSize = (16, 16)
    blockStride = (8, 8)
    cellSize = (8, 8)
    nbins = 9
    # 初始化HOGDescriptor对象
    hog = cv2.HOGDescriptor(winSize, blockSize, blockStride, cellSize, nbins)
    # 获取第一幅图像的HOG特征描述符
    descriptors1 = []
    cnt=0
    for corner in corners1:
        (x, y) = corner
        if(y-32>=0 and y+32<self.img1.shape[0] and x-32>=0 and
x+32<self.img1.shape[1]):
            cnt+=1
            patch = self.img1[y-32:y+32, x-32:x+32]
            descriptors1.append(hog.compute(patch))
    # 获取第二幅图像的HOG特征描述符
    descriptors2 = []
    cnt=0
    for corner in corners2:
        (x, y) = corner
        if(y-32>=0 and y+32<self.img2.shape[0] and x-32>=0 and
x+32<self.img2.shape[1]):
            cnt+=1
            patch = self.img2[y-32:y+32, x-32:x+32]
            descriptors2.append(hog.compute(patch))

```

```

matches = []
tag=[]
for i, descriptor1 in enumerate(descriptors1):
    best_match = -1
    best_distance = float('inf')
    for j, descriptor2 in enumerate(descriptors2):
        distance = np.linalg.norm(descriptor1 - descriptor2,2)
        if distance < best_distance and (j not in tag):
            best_match = j
            tag.append(j)
            best_distance = distance
    if best_match!=-1:
        matches.append((i, best_match, best_distance))
# 按照距离从小到大排序
matches = sorted(matches, key=lambda x: x[2])
a=[cv2.DMatch(m[0], m[1], m[2]) for m in matches]
img_matches = cv2.drawMatches(self.img1, kp1, self.img2, kp2,
a[:50],None,flags=cv2.DrawMatchesFlags_NOT_DRAW_SINGLE_POINTS)
cv2.imwrite('results/1/10000_50.png', img_matches)

```

4-3 图像拼接

图像拼接方面，主要由三个函数负责：

- `stitch()` 函数：对两张图片进行关键点检测、特征提取、匹配、透视变换和拼接，并返回结果；
- `detect_compute_plus()` 函数：对图像进行SIFT特征检测和提取，并返回关键点坐标和特征描述子；
- `match_kps()` 函数：对两张图像的特征进行匹配，利用RANSAC求解仿射变换矩阵，返回匹配点列表、变换矩阵、状态。

具体流程为，首先对两张图片进行关键点检测和特征提取，然后对两张图片的特征进行匹配，最后计算透视变换，并拼接两张图片并保存。关键代码如下（以SIFT作为特征子拼接为例）：

```

def stitch(self, ratio=0.75, Thresh=4.0):
    # 对两张图片进行关键点检测和特征提取
    kps1, des1 = self.detect_compute_plus(self.img1)
    kps2, des2 = self.detect_compute_plus(self.img2)
    # 对两张图片的特征进行匹配
    matches, H, status = self.match_kps(kps1, kps2, des1, des2, ratio, Thresh)
    # 计算透视变换，并拼接两张图片
    result = cv2.warpPerspective(self.img1, H, (self.img1.shape[1] +
self.img2.shape[1], self.img1.shape[0]))
    result[0:self.img2.shape[0], 0:self.img2.shape[1]] = self.img2
    # 将拼接结果保存到文件
    cv2.imwrite("results/1/uttower_stitching_sift.png", result)
    return result

def detect_compute_plus(self, img):
    # 将图像转为灰度图
    gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
    # 创建SIFT检测器
    descriptor = cv2.SIFT_create()
    # 在图像上检测关键点并提取特征
    (kps, features) = descriptor.detectAndCompute(img, None)
    # 将关键点的坐标转为numpy数组

```

```

kps = np.float32([kp.pt for kp in kps])
return kps, features

def match_kps(self, kps1, kps2, des1, des2, ratio, Thresh):
    # 创建BFMatcher对象
    matcher = cv2.BFMatcher()
    # 对两张图像的特征进行匹配，得到匹配结果列表
    match_lists = matcher.knnMatch(des1, des2, 2)
    # 进行筛选
    matches = []
    for m in match_lists:
        if len(m) == 2 and m[0].distance < m[1].distance * ratio:
            matches.append((m[0].trainIdx, m[0].queryIdx))
    # 根据匹配结果，计算变换矩阵，并返回匹配点列表、变换矩阵、状态
    if len(matches) > 4:
        ptsA = np.float32([kps1[i] for (_, i) in matches])
        ptsB = np.float32([kps2[i] for (i, _) in matches])
        (H, status) = cv2.findHomography(ptsA, ptsB, cv2.RANSAC, Thresh)
        return matches, H, status

```

若要实现以HOG作为特征子拼接，则只需要将 `matches` 列表换成 `HOG_kps_match()` 函数中生成的 `matches` 列表即可，在此就不再赘述。

4-4 多张图像拼接

在两张图像拼接的基础上进行多张图像的拼接，只需将图像两两拼接最后再拼接到一起即可，但需要注意的是，两张图像拼接后的新图像会有部分全黑，所以在进行二次拼接之前，需要将黑色部分裁剪掉。在这里，我采取的方法是计算图像每一列的和，若该和小于某一数值，则标记为全黑列，从这一列开始进行裁剪，具体代码如下：

```

# 预处理函数，去掉合成图像的黑边
def preprocess(path, spath, thre=10000):
    img = cv2.imread(path)
    gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

    column_sum = np.sum(gray, axis=0).reshape(-1,1)
    black_column_indices=0
    print(column_sum.shape)
    for i in range(column_sum.shape[0]):
        #print(column_sum[i][0])
        if column_sum[i][0]>thre:
            black_column_indices+=1
    img2=img[:,0:black_column_indices-1]
    cv2.imwrite(spath, img2)

```

在本次实验中，首先将yosemite1.png和yosemite2.png进行拼接，裁剪掉黑色部分，得到res1.png，然后将yosemite3.png和yosemite4.png进行拼接，裁剪掉黑色部分，得到res2.png，最后拼接res1.png和res2.png即可。

5. 结果分析与对比

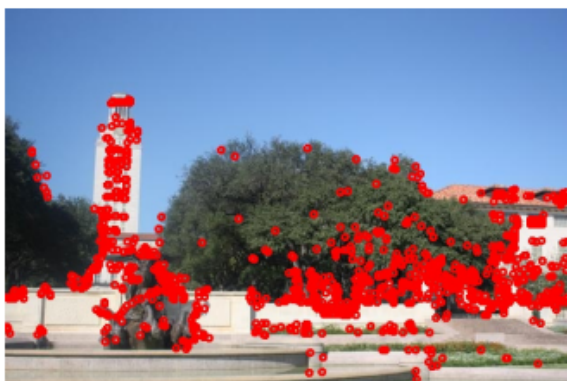
对 `images/` 目录下的 `sudoku.png` 图像进行角点检测的结果

下图左边是原图像，右边是进行角点检测标注后的图像，可见角点被准确地检测到：



Harris角点检测算法提取的 `images/1/uttower1.jpg` 和 `images/1/uttower2.jpg` 的关键点

采用Harris角点检测算法提取Harris角点检测算法提取的 `images/1/uttower1.jpg` 和 `images/1/uttower2.jpg` 的关键点，并进行绘制，结果如下图所示，上方是原图像，下方是进行关键点提取，标注后的图像：



两幅图像之间的关键点匹配的情况：

当使用SIFT特征作为描述子获得两幅图像的关键点的特征时，得到的关键点匹配图如下：



上图展示的是所有关键点，数量过多以至于难以观察，下图展示匹配度前50的关键点：



可见使用SIFT特征作为描述子获得两幅图像的关键点的特征时，关键点匹配效果良好，可以准确地找到两个图像的共同特征点。

当使用HOG特征作为描述子获得两幅图像的关键点的特征，得到的关键点匹配图如下：



图像的拼接：

SIFT特征拼接结果如下图所示，可见两张图像能够正确地拼接在一起，生成一个全景图。



HOG 特征拼接结果如下图所示，可见差别不大，但在连接处的拼接的效果没有SIFT特征拼接的好：

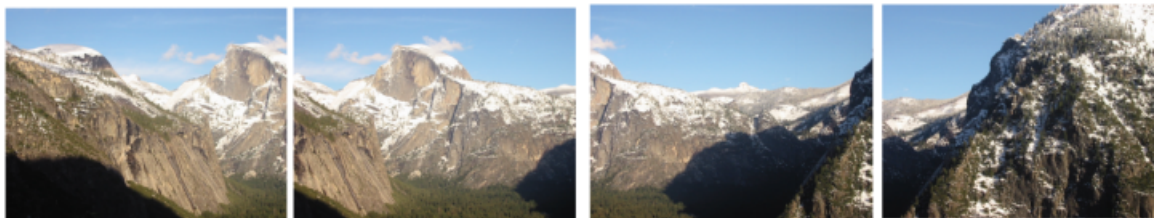


SIFT 特征和 HOG 特征在关键点匹配过程中的差异：

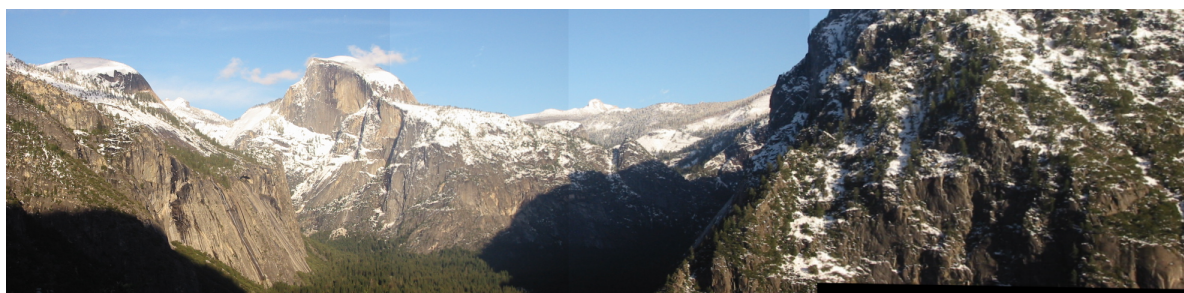
从两者的关键点匹配图可以看出，SIFT特征在关键点匹配的任务上表现要更好，这是因为HOG算法通常用来描述整个图像，而SIFT 特征更适用于关键点检测和匹配。此外，HOG作为特征描述子来进行关键点匹配需要的运行时间要更长。

基于 SIFT + RANSAC 的拼接方法用到多张图像上：

原图像：



拼接结果如下所示，可见达到了预期的拼接效果：



6. 感想与总结

在这次实验中，我使用了 Harris 焦点检测器来寻找关键点，并构建了描述算子来描述每个关键点。我还比较了两幅图像的两组描述子，并进行了匹配。根据匹配关键点，我使用了 RANSAC 算法计算仿射变换矩阵，然后将第二幅图像变换过来并覆盖在第一幅图像上，最终拼接成一个全景图像。

在实验的过程中我也遇到了许多困难，如对opencv的操作不熟练、不是很清楚如何利用HOG描述子来进行关键点匹配等。但在查找资料解决问题的过程中，我学到了许多新的知识，最后看到图像正确地拼接到一起，瞬间有了巨大的成就感。

总而言之，这次实验让我及时复习了课上所学的理论知识，并能将其运用于实际问题中，还让我熟悉了opencv的使用，RANSAC 算法的实现，这让我受益匪浅。